CS3500 Operating Systems
Lab 3: Traps

Due: 11:59 PM, 10th September, 2026

---

1 Introduction

This lab explores how system calls are implemented using traps. You will first do a warm-up exercise with stacks, and then implement an example of user-level trap handling.

Background reading

Before you start coding, read Chapter 4 of the xv6 book (`https://mit-pdos.github.io/xv6-riscv-book/`), and the related source files:

- `kernel/trampoline.S`: the assembly involved in changing from user space to kernel space and back.
- `kernel/trap.c`: code handling all interrupts.

2 Setting Up

1. Clone the course xv6 repository (or, if you already have it from the Utilities and System Calls labs, simply pull the latest changes):

```
$ git clone https://github.com/anupamsworld/IITM_CS3500_Xv6_Lab_2026.
   git
```

   The relevant directory for this lab is `traps`.

2. If you have not already done so, install Docker for your platform (Docker Desktop for Windows/macOS, Docker Engine for Linux), and pull the xv6-tools image:

```
$ docker pull nandhagk/xv6-tools:latest
```

3. Start the container with the `traps` directory mounted:

```
$ docker run -it -v <path-to-IITM_CS3500_Xv6_Lab_2026-repo>/traps:/
   home/xv6-labs nandhagk/xv6-tools:latest
```

4. Build and run xv6 to confirm everything works:

```
bash-5.3# cd /home/xv6-labs && make qemu
```

   You should see `init: starting sh` followed by a `$` prompt. Quit qemu with `Ctrl+a` followed by `x`.

3 Backtrace

For debugging it is often useful to have a backtrace: a list of the function calls on the stack above the point at which an error occurred. To help with backtraces, the compiler generates machine code that maintains a stack frame on the stack corresponding to each function in the current call chain. Each stack frame consists of the return address and a frame pointer to the caller's stack frame. Register `s0` contains a pointer to the current stack frame (it actually points to the address of the saved return address on the stack plus 8). Your backtrace should use the frame pointers to walk up the stack and print the saved return address in each stack frame.

Implement a `backtrace()` function in `kernel/printf.c`. Insert a call to this function in `sys_pause` (or the appropriate system pause routine in the kernel), and then run `bttest`, which calls `sys_pause`. Your output should be a list of return addresses of the form (the numbers will likely be different):

```
backtrace:
0x0000000080002cda
0x0000000080002bb6
0x0000000080002898
```

After `bttest` exits, quit qemu. In a terminal window, run `addr2line -e kernel/kernel` and paste in the addresses from your backtrace, e.g.:

```
$ addr2line -e kernel/kernel
0x0000000080002de2
0x0000000080002f4a
0x0000000080002bfc
Ctrl-D
```

You should see output resembling:

```
kernel/sysproc.c:74
kernel/syscall.c:224
kernel/trap.c:85
```

Hints for Backtrace Implementation

- Add the prototype for your `backtrace()` to `kernel/defs.h` so that you can invoke it from `sys_pause`.
- GCC stores the frame pointer of the currently executing function in register `s0`. In the section marked by `#ifndef ASSEMBLER ... #endif`, add the following function to `kernel/riscv.h`:

```
static inline uint64
r_fp()
{
  uint64 x;
  asm volatile("mv %0, s0" : "=r" (x) );
  return x;
}
```

  and call this function in `backtrace` to read the current frame pointer.

- The return address lives at a fixed offset (−8) from the frame pointer of a stack frame, and the saved frame pointer lives at a fixed offset (−16) from the frame pointer.
- Your `backtrace()` will need a way to recognize that it has seen the last stack frame and should stop. A useful fact: the memory allocated for each kernel stack consists of a single page-aligned page, so all the stack frames for a given stack are on the same page. You can use `PGROUNDDOWN(fp)` (see `kernel/riscv.h`) to identify the page that a frame pointer refers to.
- Once your backtrace is working, call it from `panic` in `kernel/printf.c` so that the kernel's backtrace is printed when it panics.

4 User-Level Alarm Handlers

In this exercise you will add a feature to xv6 that periodically alerts a process as it uses CPU time. This might be useful for compute-bound processes that want to limit how much CPU time they consume, or for processes that want to compute but also take some periodic action. More generally, you will be implementing a primitive form of user-level interrupt/fault handlers; a similar mechanism could be used to handle page faults in an application, for example. Your solution is correct if it passes `alarmtest` and `usertests -q`.

You should add a new `sigalarm(interval, handler)` system call. If an application calls `sigalarm(n, fn)`, then after every `n` "ticks" of CPU time that the program consumes, the kernel should cause the application function `fn` to be called. When `fn` returns, the application should resume where it left off. A tick is a fairly arbitrary unit of time in xv6, determined by how often the hardware timer generates interrupts. If an application calls `sigalarm(0, 0)`, the kernel should stop generating periodic alarm calls.

You will find a file `user/alarmtest.c` in your xv6 repository. Add it to the `Makefile`. It will not compile correctly until you have added the `sigalarm` and `sigreturn` system calls.

4.1 Phase 1: Invoke Handler (test0)

Get started by modifying the kernel to jump to the alarm handler in user space, which will cause `test0` to print "alarm!". Don't worry yet about what happens after the "alarm!" output; it is OK for now if your program crashes after printing "alarm!".

Hints for Alarm Phase 1

- You will need to modify the `Makefile` to cause `alarmtest.c` to be compiled as an xv6 user program.
- The right declarations to put in `user/user.h` are:

```
int sigalarm(int ticks, void (*handler)());
int sigreturn(void);
```

- Update `user/usys.pl` (which generates `user/usys.S`), `kernel/syscall.h`, and `kernel/syscall.c` to allow `alarmtest` to invoke the `sigalarm` and `sigreturn` system calls.

- For now, your `sys_sigreturn` should just return zero.
- Your `sys_sigalarm()` should store the alarm interval and the pointer to the handler function in new fields in the `proc` structure (in `kernel/proc.h`).
- You will need to keep track of how many ticks have passed since the last call to a process's alarm handler; you will need a new field in `struct proc` for this too. You can initialize `proc` fields in `allocproc()` in `proc.c`.
- Every tick, the hardware clock forces an interrupt, which is handled in `usertrap()` in `kernel/trap.c`.
- You only want to manipulate a process's alarm ticks if there is a timer interrupt; you want something like:

```
if(which_dev == 2) { ... }
```

- Only invoke the alarm function if the process has a timer outstanding. Note that the address of the user's alarm function might be 0 (e.g., in `user/alarmtest.asm`, `periodic` is at address 0).
- You will need to modify `usertrap()` so that when a process's alarm interval expires, the user process executes the handler function. When a trap on RISC-V returns to user space, what determines the instruction address at which user-space code resumes execution?
- It will be easier to look at traps with gdb if you tell qemu to use only one CPU:

```
$ make CPUS=1 qemu-gdb
```

4.2 Phase 2: Resume Interrupted Code (test1/test2/test3)

Chances are that `alarmtest` crashes in `test0` or `test1` after it prints "alarm!", or that `alarmtest` (eventually) prints "test1 failed", or that `alarmtest` exits without printing "test1 passed".

To fix this, you must ensure that, when the alarm handler is done, control returns to the instruction at which the user program was originally interrupted by the timer interrupt. You must ensure that the register contents are restored to the values they held at the time of the interrupt, so that the user program can continue undisturbed after the alarm. Finally, you should "re-arm" the alarm counter after each time it goes off, so that the handler is called periodically.

As a starting point, a design decision has been made for you: user alarm handlers are required to call the `sigreturn` system call when they have finished. Have a look at `periodic` in `user/alarmtest.c` for an example. This means you can add code to `usertrap` and `sys_sigreturn` that cooperate to cause the user process to resume properly after it has handled the alarm.

Hints for Alarm Phase 2

- Your solution will require you to save and restore registers. What registers do you need to save and restore to resume the interrupted code correctly? (Hint: it will be many.)
- Have `usertrap` save enough state in `struct proc` when the timer goes off that `sigreturn`

can correctly return to the interrupted user code.

- Prevent re-entrant calls to the handler—if a handler has not returned yet, the kernel should not call it again. `test2` tests this.
- Make sure to restore `a0`. `sigreturn` is a system call, and its return value is stored in `a0`.

5 Submitting the Lab

Assignment submissions are handled by Moodle. When you are ready to submit, run `make zipball`, which will generate `lab.zip`. Upload this zip file to the corresponding Moodle assignment.

Submission Checklist & Verification

- If you run `make zipball` and you have either uncommitted changes or untracked files, git will warn you.
- Inspect the list and make sure all files that your lab solution needs are tracked. You can track a new file using `git add <filename>`.
- Run `make grade` to ensure that your code passes all of the tests.
- Commit any modified source code before running `make zipball`.
- Rename the resulting `lab.zip` to `<ROLL_NUMBER>_OS_LAB_3_Traps.zip` and upload it to Moodle.

Challenge Exercise (Optional)

- Print the names of the functions and line numbers in `backtrace()` instead of numerical addresses.

Acknowledgements

A significant portion of this lab assignment is derived from `https://pdos.csail.mit.edu/6.828/2025/labs/traps.html`